

REPUBLIQUE DU CAMEROUN

PAIX-TRAVAIL-PATRIE

UNIVERSITÉ DE YAOUNDÉ I

DÉPARTEMENT D'INFORMATIQUE

REPUBLIC OF CAMEROON

PEACE-WORK-FATHERLAND

UNIVERSITY OF YAOUNDE I

COMPUTER SCIENCE DEPARTMENT

RAPPORT FINAL

**THÈME : « Implémentation d'un protocole
ZKP pour l'authentification sans mot de passe
»**

Membres du groupe :

1- FOTSING KENGNE DIANE IRIS : 17T2631

2- KOUGOUM FOTSING PAVEL : 21T2887

Année Académique 2025 - 2026

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	Problématique	1
1.3	Solution proposée	1
1.4	Objectifs du projet	1
1.5	Structure du rapport	1
2	État de l'art	1
2.1	L'authentification classique et ses limites	1
2.2	Le problème du logarithme discret	2
2.3	Preuves à connaissance nulle	2
2.4	Protocole de Schnorr	2
2.5	Transformation de Fiat-Shamir	2
2.6	Attaques par rejeu et contre-mesures	3
3	Méthodologie	3
3.1	Architecture générale	3
3.2	Les trois versions du protocole	3
3.3	Paramètres cryptographiques	3
3.4	Interface graphique	3
3.5	Protocole interactif	4
3.6	Protocole non interactif (Fiat-Shamir)	4
3.7	Contre-mesure anti-rejeu	4
3.8	Attaque par rejeu	4
3.9	Environnement de test	4
4	Résultats expérimentaux	5
4.1	Validation du protocole	5
4.2	Résistance aux attaques	5
4.2.1	Interception passive	5
4.2.2	Attaque par rejeu - version interactive	5
4.2.3	Attaque par rejeu - version vulnérable	5
4.2.4	Attaque par rejeu - version sécurisée	5
4.3	Performances	6
4.4	Comparaison interactif vs non interactif	6
5	Discussion	6
5.1	Interprétation des résultats	6
5.2	Limitations du protocole	6
5.2.1	Limitations mathématiques	6
5.2.2	Limitations pratiques	6
5.3	Comparaison avec d'autres approches	7
5.4	Enseignements tirés	7

6	Conclusion et perspectives	7
6.1	Résumé des travaux	7
6.2	Réponse à la problématique	7
6.3	Perspectives M2	7
6.3.1	Intégration dans WebAuthn	7
6.3.2	Résistance aux canaux auxiliaires	8
6.3.3	Double ratchet pour forward secrecy	8
A	Annexes	8
A.1	Code source	8
A.2	Structure des fichiers	8
A.3	Exemple d'exécution	9
A.4	Test des trois versions dans l'interface	9

1 Introduction

1.1 Contexte

Les mots de passe constituent la méthode d'authentification la plus répandue sur Internet. Cependant, leur utilisation massive s'accompagne de vulnérabilités bien connues : transmission sur le réseau, stockage en base de données, hameçonnage, et réutilisation sur plusieurs services. Chaque année, des millions de comptes sont compromis à cause de failles liées aux mots de passe.

1.2 Problématique

Comment permettre à un utilisateur de prouver qu'il connaît un secret (son mot de passe) sans jamais révéler ce secret sur le réseau ? Une interception passive ne devrait pas permettre à un attaquant de récupérer l'information sensible. De plus, le système doit résister aux attaques par rejeu où un adversaire capture et retransmet un message valide.

1.3 Solution proposée

Les preuves à connaissance nulle (Zero-Knowledge Proofs, ZKP) répondent exactement à ce besoin. Un protocole ZKP permet à un prouveur (le client) de convaincre un vérifieur (le serveur) qu'il connaît un secret, sans révéler aucune information sur ce secret. Le protocole de Schnorr, simple et efficace, a été choisi pour cette implémentation.

1.4 Objectifs du projet

- Implémenter le protocole de Schnorr, un ZKP simple et efficace
- Réaliser un échange interactif (client/serveur) puis non interactif (Fiat-Shamir)
- Évaluer la résistance aux attaques par interception et par rejeu
- Proposer une contre-mesure efficace contre les attaques par rejeu
- Mesurer les performances des différentes versions

1.5 Structure du rapport

Après un état de l'art sur les ZKP et le protocole de Schnorr, nous présenterons la méthodologie d'implémentation, les résultats expérimentaux, puis nous discuterons des limites et perspectives de ce travail.

2 État de l'art

2.1 L'authentification classique et ses limites

L'authentification par mot de passe repose sur la connaissance d'un secret partagé. Le serveur stocke un hash du mot de passe et le compare lors de la connexion. Cette approche présente plusieurs failles :

- **Transmission** : le mot de passe (ou son hash) circule sur le réseau, vulnérable à l'interception
- **Rejeu** : un attaquant peut capturer et rejouer le message
- **Stockage** : les bases de données de mots de passe sont régulièrement compromises

- **Hameçonnage** : l'utilisateur peut être trompé pour révéler son mot de passe

2.2 Le problème du logarithme discret

La sécurité de nombreux protocoles cryptographiques repose sur la difficulté du problème du logarithme discret. Soit un groupe cyclique d'ordre q généré par g . Étant donné $y = g^x \bmod p$, trouver x est considéré comme impossible en pratique pour des paramètres bien choisis (ex : p de 2048 bits). Cette propriété de "fonction à sens unique" est fondamentale pour la cryptographie asymétrique.

2.3 Preuves à connaissance nulle

Une preuve à connaissance nulle est un protocole interactif entre un prouveur P et un vérifieur V qui satisfait trois propriétés [3] :

1. **Complétude** : Si le prouveur connaît le secret, le vérifieur accepte avec une probabilité écrasante
2. **Solidité** : Si le prouveur ne connaît pas le secret, le vérifieur rejette avec une probabilité écrasante
3. **Connaissance nulle** : Le vérifieur n'apprend rien d'autre que la validité de l'assertion

2.4 Protocole de Schnorr

Proposé par Claus-Peter Schnorr en 1989 [1], ce protocole permet à un prouveur de prouver la connaissance d'un logarithme discret.

Soient des paramètres publics (p, q, g) avec $g^q \equiv 1 \bmod p$. Le prouveur possède une clé privée $x \in [1, q-1]$ et publie $y = g^x \bmod p$. Le protocole se déroule en quatre étapes :

1. **Engagement** : Le prouveur choisit r aléatoire, calcule $t = g^r \bmod p$ et envoie t
2. **Défi** : Le vérifieur choisit c aléatoire et l'envoie
3. **Réponse** : Le prouveur calcule $s = (r + x \cdot c) \bmod q$ et envoie s
4. **Vérification** : Le vérifieur accepte si $g^s = t \cdot y^c \bmod p$

La validité découle de l'égalité :

$$t \cdot y^c = g^r \cdot (g^x)^c = g^{r+x \cdot c} = g^s$$

2.5 Transformation de Fiat-Shamir

Pour rendre le protocole non interactif, Fiat et Shamir [2] proposent de remplacer le défi aléatoire c par un hash cryptographique :

$$c = H(t, y, \text{timestamp})$$

Le prouveur envoie alors $(t, s, \text{timestamp})$ en un seul message. Cette transformation est utilisée dans de nombreux systèmes modernes (signatures numériques, WebAuthn). L'avantage principal est la réduction de la latence réseau (un seul aller-retour au lieu de deux).

2.6 Attaques par rejeu et contre-mesures

Une attaque par rejeu consiste à capturer un message valide et à le retransmettre ultérieurement. Dans le cadre d'un protocole non interactif, cette attaque est particulièrement redoutable car le message est autosuffisant. Les contre-mesures classiques incluent :

- **Timestamp** : inclusion d'une date dans le hash
- **Nonce** : stockage des identifiants déjà utilisés
- **Fenêtre temporelle** : rejet des messages trop vieux (ex : plus de 60 secondes)

3 Méthodologie

3.1 Architecture générale

L'architecture retenue est de type client/serveur avec communication par sockets TCP. Le serveur joue le rôle de vérifieur et le client celui de prouveur. Une interface graphique unifiée permet de piloter les 3 versions du protocole.

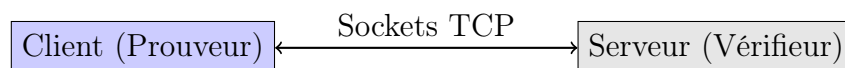


FIGURE 1 – Architecture client/serveur

3.2 Les trois versions du protocole

Version	Port	Type	Anti-rejeu
Interactive	5000	3 messages ($t \rightarrow c \rightarrow s$)	Non applicable
Fiat-Shamir vulnérable	6000	1 message (t,s,ts)	Non
Sécurisée	7000	1 message (t,s,ts)	Oui (timestamp + stockage)

TABLE 1 – Les trois versions implémentées

3.3 Paramètres cryptographiques

Pour les tests et la validation, nous utilisons de petits paramètres afin de pouvoir vérifier les calculs à la main :

Paramètre	Valeur (test)	Valeur (production)
p	23	2048 bits
q	11	256 bits
g	2	2

TABLE 2 – Paramètres cryptographiques

3.4 Interface graphique

Une interface graphique développée avec Tkinter permet de :

- Sélectionner la version du protocole (3 onglets)
- Démarrer/arrêter le serveur

- Générer les clés et s'authentifier
- Capturer et rejouer des messages pour l'attaque

3.5 Protocole interactif

Le protocole interactif suit le schéma classique de Schnorr :

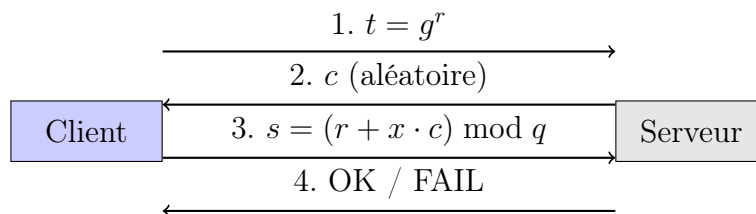


FIGURE 2 – Protocole interactif de Schnorr

3.6 Protocole non interactif (Fiat-Shamir)

La version non interactive remplace le défi par un hash SHA-256 :

Listing 1 – Calcul du défi par hash (Fiat-Shamir)

```

1 def hash_to_c(t, y, timestamp):
2     donnees = f"{t}{y}{timestamp}"
3     h = hashlib.sha256(donnees.encode()).hexdigest()
4     return int(h, 16) % Q
  
```

3.7 Contre-mesure anti-rejeu

Pour bloquer les attaques par rejeu, le serveur sécurisé implémente deux vérifications :

Listing 2 – Vérification anti-rejeu

```

1 if timestamp in timestamps_vus:
2     return "REJET - Rejeu"
3 if abs(time.time() - timestamp) > 60:
4     return "REJET - Timestamp expire"
5 timestamps_vus.add(timestamp)
  
```

3.8 Attaque par rejeu

L'interface permet de capturer un message légitime $(t, s, timestamp)$ puis de le rejouer. Selon la version du serveur : - Version vulnérable : l'attaque réussit (le serveur accepte) - Version sécurisée : l'attaque échoue (timestamp déjà utilisé ou trop vieux)

3.9 Environnement de test

- Langage : Python 3.11
- Bibliothèques : pycryptodome, tkinter (interface)
- Communication : sockets TCP
- Machine : Intel Core i5, 8 Go RAM
- 1000 échantillons par mesure de performance

4 Résultats expérimentaux

4.1 Validation du protocole

Les tests unitaires confirment le bon fonctionnement du protocole. Une authentification réussie produit le résultat suivant :

Listing 3 – Authentification réussie (côté serveur)

```
1 Serveur démarre sur 127.0.0.1:7000
2 Client connecte: ('127.0.0.1', 45564)
3 Cle publique recue : y = 18
4 Message reçu: t=13, s=5, timestamp=1744221123.456
5 Authentification reussie
```

4.2 Résistance aux attaques

4.2.1 Interception passive

Une capture réseau montre que seules les valeurs t , c et s circulent. Le secret x n'apparaît jamais, conformément à la propriété de connaissance nulle.

4.2.2 Attaque par rejeu - version interactive

Sur la version interactive, l'attaque est structurellement impossible car le défi c change à chaque session. Un rejeu simple ne peut pas fonctionner.

4.2.3 Attaque par rejeu - version vulnérable

Sur le serveur Fiat-Shamir non sécurisé, l'attaque par rejeu réussit :

Listing 4 – Attaque par rejeu réussie

```
1          ATTAQUE PAR REJEU
2 Valeurs capturees : t=16, s=8, timestamp=1744221123.456
3 Resultat du serveur : OK
4          ATTAQUE REUSSIE !
```

4.2.4 Attaque par rejeu - version sécurisée

Sur le serveur sécurisé, l'attaque est bloquée :

Listing 5 – Attaque par rejeu bloquée

```
1          ATTAQUE PAR REJEU
2 Resultat du serveur : REJET - Rejeu
3          ATTAQUE ECHOUE !
```


4.3 Performances

Les mesures de performance ont été effectuées sur 1000 échantillons :

Opération	Côté	Temps moyen (ms)
Génération des clés	Client	0.0023
Engagement ($t = g^r$)	Client	0.0018
Hash SHA-256	Client	0.0150
Réponse ($s = r + x \cdot c$)	Client	0.0004
Vérification	Serveur	0.0025
Total client		0.0195
Total serveur		0.0025

TABLE 3 – Temps d'exécution des opérations cryptographiques

4.4 Comparaison interactif vs non interactif

Version	Nombre de messages	Latence estimée
Interactive	3 (t, c, s)	$T + 2 \times RTT$
Non interactive (Fiat-Shamir)	1 (t, s, timestamp)	$T + RTT$

TABLE 4 – Comparaison des versions

La version non interactive permet de diviser par deux la latence réseau, au prix d'un calcul de hash supplémentaire (négligeable).

5 Discussion

5.1 Interprétation des résultats

Les mesures montrent que la charge cryptographique est très faible (moins de 0,02 ms pour l'ensemble des opérations client). Le goulot d'étranglement principal est donc la latence réseau, ce qui justifie l'intérêt de la version non interactive.

5.2 Limitations du protocole

5.2.1 Limitations mathématiques

Le protocole de Schnorr repose sur la difficulté du logarithme discret. L'avènement de l'informatique quantique remettrait en cause sa sécurité. Des alternatives post-quantiques existent (cryptographie basée sur les réseaux euclidiens).

5.2.2 Limitations pratiques

- Nécessite une horloge synchronisée pour la vérification des timestamps
- Le stockage des timestamps peut devenir problématique à grande échelle
- Sensible aux attaques par canaux auxiliaires (timing, cache, consommation électrique)

5.3 Comparaison avec d'autres approches

Méthode	Sécurité	Performance	Complexité
Mots de passe	Faible	Très élevée	Faible
ZKP (Schnorr)	Élevée	Élevée	Moyenne
WebAuthn	Très élevée	Élevée	Élevée
Biométrie	Moyenne	Élevée	Moyenne

TABLE 5 – Comparaison des méthodes d'authentification

5.4 Enseignements tirés

Ce projet a permis de comprendre concrètement le fonctionnement des preuves à connaissance nulle et leur application à l'authentification. La mise en œuvre pratique a révélé l'importance des détails d'implémentation (choix des paramètres, gestion des timestamps, gestion des sockets) pour la sécurité effective du système. L'interface graphique développée offre une expérience utilisateur fluide et permet de visualiser clairement les échanges.

6 Conclusion et perspectives

6.1 Résumé des travaux

Nous avons implémenté avec succès le protocole de Schnorr pour l'authentification sans mot de passe. Trois versions ont été développées : interactive, non interactive (Fiat-Shamir), et sécurisée contre les attaques par rejeu. Une interface graphique unifiée permet de piloter l'ensemble des fonctionnalités. Les mesures de performance montrent que l'approche est viable pour des applications réelles.

6.2 Réponse à la problématique

La preuve à connaissance nulle apporte une réponse élégante au problème de la transmission du secret sur le réseau. Le protocole de Schnorr, simple à implémenter et efficace, démontre qu'il est possible de s'authentifier sans jamais révéler son mot de passe. De plus, l'ajout de timestamps et de stockage permet de contrer efficacement les attaques par rejeu.

6.3 Perspectives M2

6.3.1 Intégration dans WebAuthn

WebAuthn (Web Authentication) est le standard actuel pour l'authentification sans mot de passe sur le web. Il repose sur la cryptographie à clé publique mais n'utilise pas explicitement les ZKP. Une perspective serait d'intégrer le protocole de Schnorr dans un framework WebAuthn pour bénéficier de ses propriétés.

6.3.2 Résistance aux canaux auxiliaires

Les canaux auxiliaires (temps de calcul, consommation électrique, émissions électromagnétiques) peuvent fuir des informations sur le secret. Une implémentation constante-time (temps d'exécution indépendant du secret) serait une amélioration significative pour renforcer la sécurité.

6.3.3 Double ratchet pour forward secrecy

Inspiré du protocole Signal, l'ajout d'un mécanisme de Double Ratchet permettrait de garantir la forward secrecy : la compromission d'une clé ne déchiffre pas les messages passés.

Références

- [1] Schnorr, C.P. (1989). *Efficient Identification and Signatures for Smart Cards*. CRYPTO 1989, Lecture Notes in Computer Science, vol 435, Springer.
- [2] Fiat, A., Shamir, A. (1986). *How to Prove Yourself : Practical Solutions to Identification and Signature Problems*. CRYPTO 1986, Springer.
- [3] Goldwasser, S., Micali, S., Rackoff, C. (1985). *The Knowledge Complexity of Interactive Proof Systems*. STOC 1985, ACM.
- [4] Diffie, W., Hellman, M. (1976). *New Directions in Cryptography*. IEEE Transactions on Information Theory, Vol. 22, No. 6.
- [5] Bellare, M., Rogaway, P. (1992). *Random Oracles are Practical : A Paradigm for Designing Efficient Protocols*. CCS 1993, ACM.
- [6] W3C (2019). *Web Authentication : An API for accessing Public Key Credentials*. W3C Recommendation.

A Annexes

A.1 Code source

Le code source complet est disponible sur GitHub : <https://github.com/PavelFotsing/projet-zkp-schnorr>

A.2 Structure des fichiers

```

1  Projet_ZKP/
2      interface.py          # Interface graphique (3 onglets)
3      params.py             # Param tres publics (p, q, g)
4      crypto.py             # Crypto version interactive
5      crypto_fs.py          # Crypto version Fiat-Shamir
6      serveur_auth.py       # Serveur interactif
7      client_auth.py        # Client interactif
8      serveur_fs.py         # Serveur Fiat-Shamir (vuln rable)
9      client_fs.py          # Client Fiat-Shamir
10     serveur_secure.py     # Serveur s curis (anti-rejeu)

```

```
11 client_secure.py      # Client sécurisé
12 pirate_rejeu.py       # Attaque par rejeu
13 mesures_perf.py       # Mesures de performance
```

A.3 Exemple d'exécution

Pour lancer l'interface graphique :

```
1 python3 interface.py
```

Pour lancer le serveur sécurisé en ligne de commande :

```
1 python3 serveur_secure.py
```

Pour simuler l'attaque par rejeu :

```
1 python3 pirate_rejeu.py
```

A.4 Test des trois versions dans l'interface

1. Onglet "Interactive" (port 5000) : authentification classique
2. Onglet "Fiat-Shamir" (port 6000) : authentification + attaque réussie
3. Onglet "Sécurisé" (port 7000) : authentification + attaque bloquée